

Two-Level XGBoost IDS: Training Report

A step-by-step account of a two-level classifier. Level 1 is a binary XGBoost that decides attack or not. Only flows it calls attack go to level 2, a multi-class XGBoost that names which of the 14 attacks it is. This report covers the data, every parameter, how each level was trained and tested, and what the results mean.

Overall accuracy 96.3% 15-class, end to end	Classes >= 95% 9 / 15 recall	Level 1 : binary attack vs not	Level 2 : 14-class which attack	Test rows 465,837 unseen in training
--	---	---	--	---

Headline. The two-level model reaches 96.3% overall accuracy, with 9 of 15 classes at 95% recall. Level 1 catches ~94.8% of attacks and passes them to level 2, which names the big attack types perfectly. The misses are Infiltration (looks like normal traffic) and four classes with only 8-45 test rows.

01 — ARCHITECTURE

Two levels, both XGBoost

- Level 1 - binary XGBoost (attack vs not).** Every non-Benign label is treated as one class, "attack". If the flow is called not-attack, the answer is Benign and it stops here. If attack, it is passed on.
- Level 2 - multi-class XGBoost (which attack).** Only for flows level 1 flagged. Trained on attack rows only (Benign removed, because level 1 already filtered it), it names one of the 14 attack types.

The final label is Benign when level 1 says not-attack, otherwise the attack level 2 chose. An attack that level 1 wrongly calls not-attack never reaches level 2, so the pipeline can never catch more attacks than level 1 passes on.

02 — DATA & SPLIT

Same rows and split for both levels

Source: **cicids2018_full_cleaned.csv** (11,872,582 cleaned flows, 79 columns). Every attack row is used, plus a random 1,000,000 Benign rows (the full 10.5M do not fit in 12 GB of RAM). The CSV is read in 200,000-row chunks and saved as NumPy arrays, so the later steps memory-map them and never re-read the 4.8 GB file.

Each class is split **80% train / 20% test** (stratified, seed 42), no validation set. Both levels use the same split, so the 465,837 test rows are unseen by both models.

Working set 2,329,184 rows -> train 1,863,347, test 465,837. Level 1 trains on all training rows; level 2 trains on the 1,063,659 attack rows of the training split only.

03 — LEVEL 1: ATTACK vs NOT

Binary XGBoost, the gate

A gradient-boosted tree model. It builds trees in rounds; each new tree corrects the errors of the ones before it. Every flow gets a probability of being an attack; above 0.5 it is passed to level 2. Balanced sample weights stop the Benign majority (about 46% of the working set) from dominating.

Parameter	Value	Meaning
objective	binary: logistic	attack (1) vs Benign (0)
n_estimators	500	boosting rounds / trees

Parameter	Value	Meaning
learning_rate	0.1	step size per tree
max_depth	10	tree depth
subsample	0.8	rows per tree
colsample_bytree	0.8	features per tree
tree_method	hist	fast histogram training
sample_weights	balanced	up-weights the rarer side
random_state	42	reproducible

Training fit: accuracy 96.8%, attack precision 99.5%, attack recall 94.8%. So level 1 catches about 95 of every 100 attacks while almost never mislabelling normal traffic - a far stronger gate than an unsupervised anomaly detector.

04 — LEVEL 2: WHICH ATTACK

Multi-class XGBoost on attack rows only

Same tree settings as level 1, but `objective=multi:softprob` over 14 attack classes and square-root balanced weights (rare attacks count more). Benign is excluded, so level 2 only ever decides between attack types.

Training fit: 99.99% accuracy. Every large attack is named at 100%. Only the tiny classes are weak even in training - DoS-SlowHTTPTest 50.0% (34 rows) and FTP-BruteForce 77.4% (31 rows) - because so few rows remain after duplicate removal.

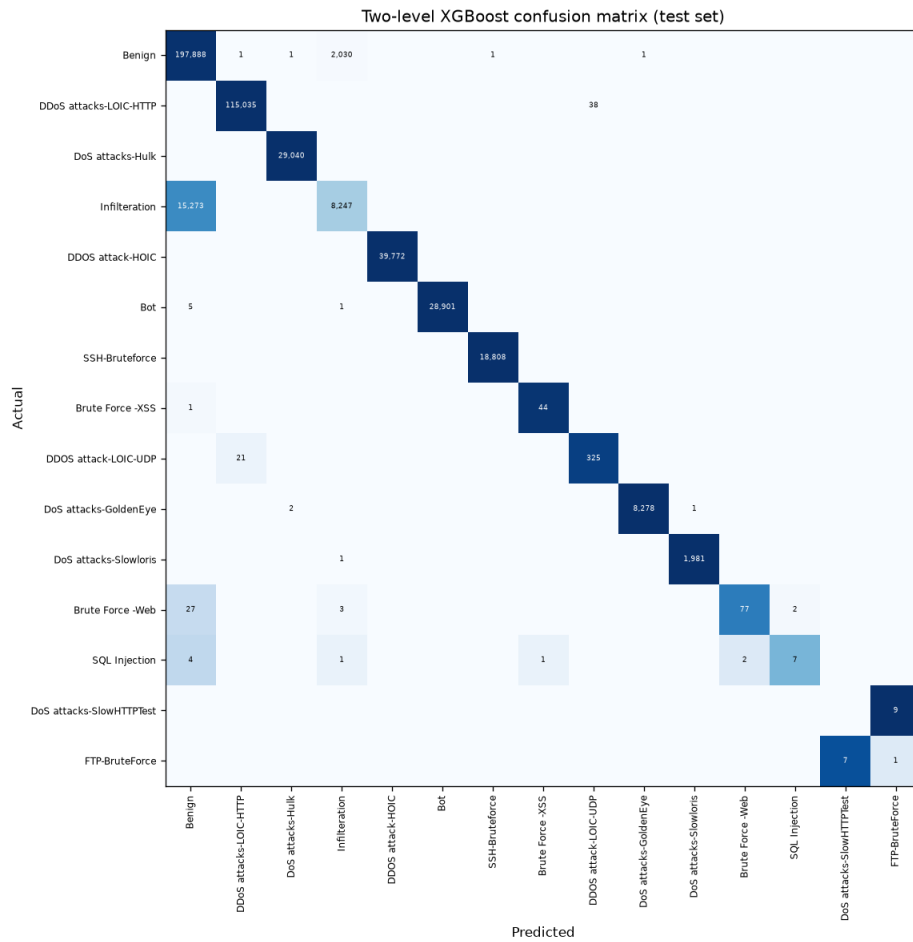
```
final_label = Benign          if level 1 says not-attack
                = level 2's attack name if level 1 says attack
```

05 — PIPELINE RESULTS

Both levels together on the test set

class	test rows	precision	recall	F1
Benign	199,922	92.8%	99.0%	0.958
DDoS attacks-LOIC-HTTP	115,073	100.0%	100.0%	1.000
DDoS attack-HOIC	39,772	100.0%	100.0%	1.000
DoS attacks-Hulk	29,040	100.0%	100.0%	1.000
Bot	28,907	100.0%	100.0%	1.000
Infiltration	23,520	80.2%	35.1%	0.488
SSH-Bruteforce	18,808	100.0%	100.0%	1.000
DoS attacks-GoldenEye	8,281	100.0%	100.0%	1.000
DoS attacks-Slowloris	1,982	99.9%	99.9%	0.999
DDoS attack-LOIC-UDP	346	89.5%	93.9%	0.917
Brute Force -Web	109	97.5%	70.6%	0.819
Brute Force -XSS	45	97.8%	97.8%	0.978
SQL Injection	15	77.8%	46.7%	0.583
DoS attacks-SlowHTTPTest	9	0.0%	0.0%	0.000
FTP-BruteForce	8	10.0%	12.5%	0.111
Overall	465,837		96.3%	

Recall = attacks caught by level 1 AND named correctly by level 2. Green = 95%+, 9 of 15 reach it. Low recall with high precision (Infiltration 35% / 80%) means level 1 missed the attacks; low precision (FTP 10%) means the tiny class is confused with another.



06 — MEASURES

What each number means

measure	meaning
Accuracy	share of all test flows given the correct label
Recall	of a class's real flows, the share correctly caught (detection rate)
Precision	of flows predicted as a class, the share that really are it
F1	balance of precision and recall

07 — CONCLUSION

What this shows

- **96.3% overall, 9 of 15 classes at 95% recall.** A strong, clean two-level design: a binary gate then an attack namer.
- **Level 1 is a strong gate.** It catches ~94.8% of attacks with 99.5% precision, so few normal flows are flagged and most attacks reach level 2.
- **Level 2 names the big attacks perfectly.** LOIC-HTTP, HOIC, Hulk, Bot, SSH, GoldenEye and Slowloris are all at ~100%.
- **Six misses, from two data limits.** Infiltration (35%) looks like normal traffic, so level 1 does not flag it. SQL Injection, FTP-Bruteforce and DoS-SlowHTTPTest have only 8-15 test rows after duplicate removal, too few to learn or measure. Brute Force -Web (71%) is partly both.

- **To lift the six:** merge the tiny web/DoS classes into families, or accept that Infiltration and the few-row classes cannot reach 95% on this dataset.

Scripts: multi_xgboost/1_prepare_data.py ... 4_evaluate.py. Numbers from final_metrics.json, level1_report.txt, level2_report.txt. Compiled 2026-09-29.